```
import { ThunkAction } from "redux-thunk"
import { createConsumer } from "@rails/actioncable"
import { TypedUseSelectorHook, useDispatch, useSelector } from "react-redux"
import { Subscription } from "@rails/actioncable"
```

There's one complication. Redux doesn't want our ActionCable subscription to be part of the reducer's state because it's not serializable. This means that we need to maintain our subscription globally the way we did before.

chapter_12/02/app/javascript/contexts/venue_context.ts

```
let subscription: Subscription

export const initSubscription = (): void => {
  if (subscription === undefined) {
    subscription = createConsumer().subscriptions.create(
      {
        channel: "ConcertChannel",
        concertId: venueStore.getState().concertId,
      },
      {
        received(tickets) {
          venueStore.dispatch({ type: "setTickets", tickets })
        },
      }
    )
  }
}
```

This initSubscription function encapsulates the same subscription behavior we've seen before.

Our code here guards against re-creating the subscription if it's already there. If the subscription is not there, then we have the same creation code as before, using the dispatch function to set the venue data.

We can now move our API-based data fetch to a thunk:

chapter_12/02/app/javascript/contexts/venue_context.ts

```
type VenueThunk = ThunkAction<void, VenueState, null, VenueAction>
```